



15. Python: Mathematical Functions

"Use Python to solve math problems. That's about it"

Previous Section:

 [14. Python: Strings](#)

Contents:

[1. The `math` Library](#)

[2. Some Common Mathematical Functions](#)

[Built-in Mathematical Functions](#)

[2. Functions from the `math` Library](#)

[Number-Theoretic and Rounding Functions](#)

[Power, Exponential, and Logarithmic Functions](#)

[Trigonometric and Angular Conversion Functions](#)

[Mathematical Constants](#)

[3. Alternative Advanced Math Libraries](#)

[Summary](#)

[References](#)

Python is versatile, and one of the areas where it truly shines is mathematics. Whether we are working with simple arithmetic, geometry, logarithms, or trigonometry, Python already provides many built-in tools to make our lives easier.

In this section, we will focus on the basics using Python's native mathematical features and the `math` library. Surprisingly, the `math` module alone can already solve a large portion of common mathematical problems. Later on, if we want to work with advanced scientific computing, machine learning, or symbolic algebra, we can expand into libraries such as NumPy, SciPy, and SymPy.

1. The `math` Library

Python provides a standard library called `math`. Since it is part of Python's standard library, we do not need to install anything separately.

```
import math
```

Another common way is importing only the functions we need:

```
from math import sqrt, factorial
```

I personally use this method when I only need a few specific functions because it keeps the code shorter and cleaner. It can also slightly reduce unnecessary memory usage, although in most beginner programs the difference is very small.

We will discuss Python libraries and importing strategies more deeply in another section.

2. Some Common Mathematical Functions

Built-in Mathematical Functions

Before using the `math` library, Python already provides several useful mathematical functions.

- `abs(x)` : Returns the absolute value of a number.

```
print(abs(-15))
```

15

- `divmod(x, y)` : Returns both the quotient and remainder as a tuple.

```
print(divmod(17, 5))
```

(3, 2)

- `max(iterable)` and `min(iterable)` : Find the largest and smallest values.

```
numbers = [5, 1, 9, 3]
```

```
print(max(numbers))
```

```
print(min(numbers))
```

9

1

- `pow(x, y)` : Raises a number to a power.

```
print(pow(2, 5))
```

32

- `round(x, n)` : Rounds a number.

```
print(round(3.141592, 2))
```

3.14

- `sum(iterable)` : Adds all elements together.

```
numbers = [1, 2, 3, 4]

print(sum(numbers))
```

10

- Another very useful mathematical and functional programming tool in Python is the `map()` method. The `map()` function applies another function to every element inside an iterable (such as a list or tuple) and returns the transformed results.

The syntax is:

```
map(function, iterable)
```

For example, imagine we want to square every number inside a list.

```
num_list = [1, 2, 3, 4, 5, 5, 8, 9, 9, 10]

def square(x):
    return x ** 2

squared_list = list(map(square, num_list))
print(squared_list)
```

[1, 4, 9, 16, 25, 25, 64, 81, 81, 100]

What happens here is actually very straightforward:

- `map()` takes the `square()` function
- It applies the function to every element in `num_list`
- The results are returned as a map object
- We convert it into a list using `list()`

You can think of it like this:

```
1 → square(1) → 1
2 → square(2) → 4
3 → square(3) → 9
...
```

We can also combine `map()` with `lambda` functions for shorter code:

```
num_list = [1, 2, 3, 4, 5]

squared_list = list(map(lambda x: x ** 2, num_list))
print(squared_list)
```

[1, 4, 9, 16, 25]

The `map()` method is especially useful when we want to transform data efficiently without writing manual loops.

2. Functions from the `math` Library

Number-Theoretic and Rounding Functions

- `math.ceil(x)` : Rounds upward to the nearest integer.

```
import math
print(math.ceil(4.2))
```

5

- `math.floor(x)` : Rounds downward to the nearest integer.

```
print(math.floor(4.9))
```

4

- `math.comb(n, k)` : Computes combinations.

```
print(math.comb(5, 2))
```

10

- `math.factorial(n)` : Computes factorial.

```
print(math.factorial(5))
```

120

- `math.gcd(x, y)` : Finds the greatest common divisor.

```
print(math.gcd(24, 36))
```

12

- `math.isclose(a, b)` : Checks if two floating-point numbers are approximately equal.

```
print(math.isclose(0.1 + 0.2, 0.3))
```

True

- `math.prod(iterable)` : Multiplies all elements together.

```
print(math.prod([1, 2, 3, 4]))
```

24

Power, Exponential, and Logarithmic Functions

- `math.sqrt(x)` : Returns the square root ($y = \sqrt{x}$)

```
print(math.sqrt(25))
```

5.0

- `math.cbrt(x)` : Returns the cube root.

```
print(math.cbrt(27))
```

3.0

- `math.exp(x)` : Computes e^x ($y = e^x$).

```
print(math.exp(2))
```

7.38905609893065

- `math.log(x, base)` : Returns logarithms.

```
print(math.log(8, 2))
```

3.0

- `math.log10(x)`

```
print(math.log10(1000))
```

3.0

- `math.log2(x)`

```
print(math.log2(16))
```

4.0

Trigonometric and Angular Conversion Functions

┆ Note: Trigonometric functions in Python use radians, not degrees.

- `math.sin(x)`, `math.cos(x)`, `math.tan(x)`

```
print(math.sin(math.radians(90)))
print(math.cos(math.radians(90)))
print(math.tan(math.radians(90)))
# Cotane
print(1 / math.tan(math.radians(90)))

# Special Cases
print(math.sin(math.pi / 2))
print(math.cos(math.pi))
print(math.tan(math.pi / 4))
```

1.0

6.123233995736766e-17

1.633123935319537e+16

6.123233995736766e-17

1.0

1.0

0.9999999999999999



You may notice that `tan(pi / 4)` is not exactly `1`. This happens because computers store floating-point numbers approximately, not perfectly.

- Inverse Trigonometric Functions

```
print(math.asin(1))  
print(math.acos(1))  
print(math.atan(1))
```

```
1.5707963267948966  
0.0  
0.7853981633974483
```

- `math.atan2(y, x)`

```
print(math.atan2(1, 1))
```

```
0.7853981633974483
```

- `math.degrees(x)` and `math.radians(x)`

```
print(math.degrees(math.pi))  
print(math.radians(180))
```

```
180.0  
3.141592653589793
```

- `math.hypot(*coordinates)`

Computes the hypotenuse.

```
print(math.hypot(3, 4))
```

```
5.0
```

This is closely related to the Pythagorean Theorem: $a^2 + b^2 = c^2$

Mathematical Constants

The `math` module also provides several important mathematical constants.

- `math.pi`

```
print(math.pi)
```

3.141592653589793

- `math.e`

```
print(math.e)
```

2.718281828459045

- `math.tau`

```
print(math.tau)
```

6.283185307179586

- `math.inf`

```
print(math.inf)
```

inf

- `math.nan`

```
print(math.nan)
```

nan

3. Alternative Advanced Math Libraries

Eventually, we may outgrow the standard `math` module. When that happens, Python offers several powerful mathematical ecosystems.

- **NumPy**: Excellent for arrays, matrices, vectorized operations, and high-performance numerical computing.
- **SciPy**: Built on top of NumPy and commonly used for scientific engineering, optimization, integration, and simulations.
- **SymPy**: Designed for symbolic mathematics such as algebra simplification, calculus, and solving equations symbolically instead of numerically.

These libraries are extremely important in fields like Data Science, Artificial Intelligence, Engineering, and Scientific Computing.

Summary

In this section, we explored Python's mathematical capabilities through both built-in functions and the `math` library. We learned that Python already provides many tools for arithmetic, logarithms, trigonometry, factorials, rounding, and constants without requiring external installations.

Even though the standard `math` module is already powerful, it is only the beginning. As we move further into programming, data science, machine learning, or scientific computing, libraries such as NumPy, SciPy, and SymPy will expand what we can do far beyond basic calculations.

References

<https://docs.python.org/3/library/math.html>

<https://www.geeksforgeeks.org/python/python-math-module/>

<https://www.programiz.com/python-programming/modules/math>

Next Section:

[`</>` 16. Python: Date and Time Functions](#)